

Name: Rhilo Sotto  
RED ID: 130551574  
Date: 9/13/2024

# Lab 2 Report

## 1. What does your program do?

The program transmits my RedID character by character in USART within an infinite loop. The program runs continuously, and since there is no break statement, it will run indefinitely until power is lost. The signal is received by a separate breakout board viewable in PuTTY.

## 2. What variables/registers does your program use?

The program uses the **UBRR0H** and **UBRR0L** registers to set the BAUD RATE using the UBBR equation (high stores 8 high bits, and low stores 8 low bits) which is calculated beforehand.

The **UCSR0B** register enables the receiver (RXEN0 - pin 3 = 1) and transmitter (TXEN0 - pin 4 = 1) of the microcontroller (only the transmitter is used in this lab).

The **UCSR0C** register to set the microcontroller to Asynchronous USART mode (UMSEL00 and UMSEL01 - pins 6 and 7 = 0), the signal having 8 data bits (UCSZ00 and UCSZ01 - pins 1 and 2 = 1), to disable the parity bit (UPM00 and UPM01 - pins 4 and 5 = 0), and signal only sending 1 stop bit (USBS0 - pin 3 = 0). The signal is 10 bits total including the start bit.

The **UCSR0A** register is used to check whether the transmit buffer is empty (UDRE0 - pin 5 = 1).

The **UDR0** register is used to transmit the data (8 bit chars).  
The variable **stringIndex** is used to iterate through each character/digit of the **redID** array and is set back to zero upon reaching the end of the string to repeat transmitting the redID.

## 3. What is the main algorithm/logic of your program?

The main program calls the `usart_init` function to initialize all of the necessary microcontroller settings, including setting the baud rate, enabling the receiver and transmitter, setting the USART mode to asynchronous, an 8 bit data frame, disabling the parity bit, and only sending 1 stop bit. Then the program has the redID initialized as a char array and an index variable to iterate through the array.

The program enters the infinite while loop, where it will check whether the string has reached the terminating null character and either transmit the character using the `usart_transmit` function and increment the index, or transmit the carriage return, a line feed, a 500 ms delay, and return back to the beginning of the string.

The `usart_transmit` function uses a busy while to wait until the transmit buffer is empty before transmitting new data in the UDR0 register.

## 4. What external hardware connections did you make?

The microcontroller is the transmitter which is hooked up via USB to my computer and the FT232R Breakout board, which is set up as a USART (Serial) Receiver using PuTTY, through the RX pin connected to the microcontroller in PD1 (TXD). Output is read through COM4 using PuTTY with a USB from the breakout board to my computer.

## Appendix (Code)

```
/*
 * Lab2.c
 *
 * Created: 9/12/2024 3:47:01 PM
 * Author : rnsot
 */

#define F_CPU 16000000UL // 16MHz clock speed

#define BAUD 9600 // bits/sec
#define BAUDRATE ((F_CPU)/(BAUD*16UL) - 1) // UBRR

#include <avr/io.h>
#include <util/delay.h> // to use delays

void usart_init (void) { // initialize AVR settings
    UBRR0H = (BAUDRATE>>8); // store 8 most significant bits of BAUDRATE
    UBRR0L = BAUDRATE; // store 8 least significant bits of BAUDRATE

    UCSR0B |= (1<<RXEN0) | (1<<TXEN0); // enable receiver and transmitter
    // only transmitter is used for this lab

    UCSR0C &= ~(1<<UMSEL00) | ~(1<<UMSEL01); // Asynchronous USART mode
    UCSR0C |= (1<<UCSZ00) | (1<<UCSZ01); // 8 data bit frame
    UCSR0C &= ~(1<<UPM01) | ~(1<<UPM00); // disable parity bit
    UCSR0C &= ~(1<<USBS0); // 1 stop bit
}
```

```

void usart_transmit(char data) { // transmit one char
    while ( !(UCSR0A & (1<<UDRE0)) ); // wait until transmit buffer is empty
    UDR0 = data; // write data to register
}

int main(void) {
    usart_init();
    char redID[] = "130551574";

    uint8_t stringIndex = 0; // iterate through each character
    while (1) {
        if (redID[stringIndex] != '\0') { // until end is reached
            usart_transmit(redID[stringIndex]);
            ++stringIndex; // next index
        }
        else {
            usart_transmit('\r'); // carriage return
            usart_transmit('\n'); // line feed
            _delay_ms(500); // 500ms delay
            stringIndex = 0; // back to beginning
        }
    }
}

```